

Appendix

A. Audio Caption Models

We selected two open-source audio captioning models based on complementary considerations:

- SLAM-AAC [1] is an LLM-based Automated Audio Captioning model which uses the EAT [2] as the audio encoder and Vicuna-7B [3] as the LLM decoder enhanced with a CLAP-Refine process. Selecting this model allows the generated captions to remain closely aligned with CLAP representations.
- MiDashengLM [4] is a 7B parameter open-source audio understanding model that uses a proprietary audio encoder and a prefix-based LLM decoder. This model was selected based on the superior performance compared to the other options both through reported performance and subjective tests over unseen samples (Most alignment with human opinion).

The prompts used for the SLAM-AAC model to instruct it for captioning the audio is as follows:

```
Prompt = "Describe the audio you hear. Output the  
↳ audio caption directly without redundant content.  
↳ Ensure that the output is not duplicated."
```

For MiDashengLM the instruction is provided by a prefixed prompt "Transcribe/Caption this audio." used while pretraining and finetuning the model. Since the model has been overly instruction-tuned with the prefixed prompt, any other form of instruction results in an output collapse and does not provide a sensible caption.

Lastly, the output of the captioning models along with the original caption for the audio are concatenated using the "|" symbol. Some examples of audio gone through all the captioning models:

```
Music Sample = "GT: playing didgeridoo | SLAM-AAC: A  
↳ person is playing a didgeridoo | MiDashengLM: A  
↳ didgeridoo plays a melody with a consistent  
↳ rhythmic clicking in the background."
```

```
Ambient Noise Sample = "GT: sizzling food with distant  
↳ mechanical humming | SLAM-AAC: Food sizzles in a  
↳ pan | MiDashengLM: A motorboat is running on  
↳ water."
```

B. LLM as a Judge for Captions

we used the open-source gpt-oss-20b [5] for the LLM-as-a-Judge component of the automated data engine pipeline. The model was provided with the definition of the sound classes through the system prompt. Along with the definitions, some

examples of possible sub-categories for each sound class was also given to model in the system prompt. The complete system prompt is as follows:

```
SYSTEM_PROMPT = (  
    """  
    You are an expert Audio Content Analyzer. Your task is  
    ↳ to determine, for a given caption,  
    whether it describes **two or more distinct sound  
    ↳ events that fall under different main sound  
    ↳ categories**.  
  
    ### MAIN SOUND CATEGORIES (Primary Reference for  
    ↳ Classification)  
    Use this list to categorize the sound events described  
    ↳ in the caption. The goal is to count how many of  
    ↳ these distinct categories are mentioned.  
  
    1. **Ambient Noise**: Includes: White noise, Static,  
    ↳ General traffic, Engines (Car, Truck, Aircraft,  
    ↳ Motorcycle), Sirens, Tools (Drills, Saws, Hammers,  
    ↳ Sanding, etc.),  
    Household items (Blender, Vacuum, Hair dryer, Clock,  
    ↳ Alarm, etc.), Traffic, Construction, Crinkling,  
    ↳ Machine running, Computer keyboard, Insects (bees,  
    ↳ fly, crickets, etc.),  
    Insects buzzing, Crying, Baby noises, Car horns,  
    ↳ Engine idling, Tire screeching, Brakes, Vehicle  
    ↳ acceleration sounds, Hissing not from animals, and  
    ↳ General hums.  
    2. **Crowd Noise**: Includes: Crowds, Chanting,  
    ↳ Applause, Cheering, Clapping, Collective humming,  
    ↳ Audience.  
    3. **Music**: Includes: Instruments (Guitar, Piano,  
    ↳ Drums, Violin, Synthesizer, etc.), Genres (Rock,  
    ↳ Jazz, Hip hop, Classical, etc.), Singing,  
    ↳ Whistling (musical), Bells, and Rhythm.  
    4. **Nature**: Includes: Water sounds (Rain, Ocean,  
    ↳ Stream, Drip, Splash, Waves), Thunder, and other  
    ↳ non-animal natural phenomena.  
    5. **Animal**: Includes: All animal vocalizations  
    ↳ (Bark, Meow, Moo, Chirp, Roar, Hoot, Growl, Tweet,  
    ↳ Oink etc.),  
    animal's physical action/movements (wings flapping,  
    ↳ hooves clip-clopping, cat's purring, dog's  
    ↳ panting, etc.), and specific animals (Dog, Cat,  
    ↳ Bird, Horse, Birds, etc.).  
    6. **Speech**: Includes: Human speech, Speak,  
    ↳ Conversation, Laughter, Screaming, Whispering,  
    ↳ Synthetic speech, Narration.  
    7. **Wind**: Includes: Whoosh, Swoosh, Rustling  
    ↳ leaves, Microphone wind noise, Wind blows, Gusts  
  
    ### EVALUATION LOGIC  
    Follow these steps in order for every caption:  
  
    1. **Identify Sound Events**: Break down the  
    ↳ caption into its individual, distinct sound  
    ↳ components.  
    * *Example*: "A bird chirping near a stream with  
    ↳ light music" -> 3 events: Bird Chirping,  
    ↳ Stream, Music.  
    * *Crucial Rule*: Descriptive modifiers or single  
    ↳ complex sound do not count as separate events  
    ↳ (e.g., 'Loud thunder' is 1 event, 'a heavy  
    ↳ engine idling' is 1 event).
```

```

2. **Classify Sound Events**: Map each identified
↳ sound event to one of the **MAIN SOUND
↳ CLASSES** above. Use your general knowledge
↳ for sounds not explicitly listed.
* *Example: Bird Chirping -> Animal. Stream ->
↳ Nature. Music -> Music.
* *Note: The same class mentioned twice (e.g., 'A
↳ dog and a cat') still only counts as 1 class
↳ (Animal).

3. **Determine 'hasMultSources'**:
* If the caption describes sound events that
↳ belong to **2 or more different MAIN SOUND
↳ CLASSES**, set 'hasMultSources' = 'true'.
* Otherwise (1 class or 0/unclear classes), set
↳ 'hasMultSources' = 'false'.

### OUTPUT FORMAT
You must output a JSON object for each item with
↳ ONLY the following fields (do not echo the
↳ input):
{
  "hasMultSources": boolean,
  "reasoning": "string (Start with 'Identified X
↳ distinct sound classes:', then list the
↳ classes/events found. Example: 'Identified 2
↳ distinct sound classes: Music and Ambient
↳ Noise (Traffic).)'"
}
"""
)

```

The prompt is then generated for a batch of captions while providing a few-shot example of captions and answers. In order to track the decision made by the LLM model, we also add the structured output format to the user prompt with the following fields: (a) *hasMultSources*, is true when the caption mentions multiple sound classes, (b) *reasoning*, contains the thought process behind the decision made by the model on whether the caption contains multiple sound classes or not. The prompt is constructed as follows:

```

# 1. Prompt Header
user_prompt = (
    "Evaluate the following audio captions. Your task
    ↳ is to determine if the caption describes "
    "**two or more distinct sound events that belong
    ↳ to DIFFERENT main sound classes** "
    "(e.g., Speech and Music, or Animal, Wind, and
    ↳ Ambient Noise). "
    "A single sound event with multiple descriptive
    ↳ words (e.g., 'Loud, continuous traffic noise')
    ↳ does NOT count as multiple sources.\n\n"
)

# 2. Few-Shot Examples (Crucial for teaching the model
↳ how to count and categorize)
user_prompt += """
### REFERENCE EXAMPLES:

1. Input: {Caption: "A cat meowing and a car driving
↳ by"}

```

```

-> Result: {"hasMultSources": true, "reasoning":
↳ "Identified 2 distinct sound classes: Animal
↳ (Cat meowing) and Ambient Noise (Car
↳ driving)."}

2. Input: {Caption: "A loud clap of thunder and heavy
↳ rain falling"}
-> Result: {"hasMultSources": false, "reasoning":
↳ "Identified 1 distinct sound class: Nature
↳ (Thunder and rain are both Nature events)."}

3. Input: {Caption: "A person singing a soft melody"}
-> Result: {"hasMultSources": false, "reasoning":
↳ "Identified 1 distinct sound class: Music
↳ (Singing is a musical event)."}

4. Input: {Caption: "Speech with background music in a
↳ car"}
-> Result: {"hasMultSources": true, "reasoning":
↳ "Identified 3 distinct sound classes: Speech,
↳ Music, and Ambient Noise (Car)."}

### ACTUAL DATA TO EVALUATE:
"""

# 3. The Data Loop
for i, caption_dasheng in
↳ enumerate(caption_dasheng_list):
    user_prompt += f"Item {i}:\n"
    user_prompt += f"\tCaption: {caption_dasheng}\n\n"

# 4. Footer (Strict Formatting & Length Constraint)
user_prompt += (
    f"Output Requirement: Return the results in the
    ↳ exact JSON format specified in the system
    ↳ prompt. "
    f"You MUST return exactly
    ↳ {len(caption_dasheng_list)} objects in the
    ↳ 'results' array, matching the order above. "
    "Do not skip any Items."
)

```

C. Synthesized Audio Samples

For the wind and crowd sound classes the number of filtered samples did not reach the minimum threshold for training. Therefore we used two methods to generate more samples for these classes.

For wind class we used a diffusion-based generative models, namely, Auffusion [6]. This model was selected based on the superior generation quality compared to other models with the specific use-case of generating wind sounds. The prompts used for generating the audio samples were:

```

"brisk wind sweeping across fields"
"calm wind barely moving the air"
"dry wind rustling autumn leaves"
"fierce wind rattling windows"
"gentle breeze rustling leaves"
"moderate wind blowing steadily"
"powerful gusts howling through canyons" *

```

```
"soft wind flowing through grass"  
"stormy wind roaring over mountains" *  
"strong wind shaking tree branches"  
"violent wind bending tall trees"  
"whistling wind around mountain peak" *
```

However, we decided to remove the prompts highlighted by "*" from the generated samples as they often contained sounds unrelated to wind such as howling, roaring, and whistling sounds which may have been rooted in the mention of these phases in the prompt. Each prompt was given to the Auffusion model with a random noise seed each time to ensure the resulting generation has as much variety as possible. A total of 10K samples (1K samples for each successful prompt) were generated for the wind class using this method.

For the crowd class, our investigation with generative models showed that the audio samples produced by the audio generation models do not have the quality required of the sound separation task. Most samples generated for crowd variations such as hub-bub, always contained instances of one human voice being prominent compared to other voices. Although the content of the prominent voice were not intelligible, the voice signature did match the speech sound class and training with those samples resulted in sound leakage from speech into separated crowd sound. To address this issue, we opted to breakdown the hub-bub sound into its components and produced sufficient samples by a 2-stage process of Mixing 7-9 recordings of clean speech, and convolving a randomly selected sample of real-world room impulse response over the mixture.

1. REFERENCES

- [1] Wenxi Chen, Ziyang Ma, Xiquan Li, Xuenan Xu, Yuzhe Liang, Zhisheng Zheng, Kai Yu, and Xie Chen, “Slam-aac: Enhancing audio captioning with paraphrasing augmentation and clap-refine through llms,” *arXiv preprint arXiv:2410.09503*, 2024.
- [2] Wenxi Chen, Yuzhe Liang, Ziyang Ma, Zhisheng Zheng, and Xie Chen, “Eat: self-supervised pre-training with efficient audio transformer,” in *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence*, 2024, IJCAI ’24.
- [3] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al., “Judging llm-as-a-judge with mt-bench and chatbot arena,” *Advances in neural information processing systems*, vol. 36, pp. 46595–46623, 2023.
- [4] Heinrich Dinkel, Gang Li, Jizhong Liu, Jian Luan, Yadong Niu, Xingwei Sun, Tianzi Wang, Qiyang Xiao, Junbo Zhang, and Jiahao Zhou, “Midashenglm: Efficient audio understanding with general audio captions,” *arXiv preprint arXiv:2508.03983*, 2025.
- [5] OpenAI, “gpt-oss-120b & gpt-oss-20b model card,” 2025.
- [6] Jinlong Xue, Yayue Deng, Yingming Gao, and Ya Li, “Auffusion: Leveraging the power of diffusion and large language models for text-to-audio generation,” *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 32, pp. 4700–4712, 2024.